Software Architecture and Design Specification

Project: Voice Assistant for Multimedia Controls

Version: 1.0

Authors: (TEAM 10)
Yash Raghavendra A (PES2UG24CS913)
Kushi Niranjana (PES2UG24CS245)
Hemant Ram Nelapati (PES2UG24CS192)
Keerthi B Patil (PES2UG24CS225)

Date: 03-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	03-10-2026	Project Team	Initial SAD for Voice Assistant for Multimedia Controls based on SRS v1.0 and Test Plan v1.0.

Approvals

Role	Name	Signature/Date
Course Coordinator		

Table of Content

Contents

Revision History 1

Approvals 1

Table of Content 2

1. Introduction 3

1.1 Purpose 3

1.2 Scope 3

1.3 Audience 3

1.4 Definitions 3

2. Document Overview 3

2.1 How to use this document 3

2.2 Related Documents 3

3. Architecture 4

3.1 Goals & Constraints 4

3.2 Stakeholders & Concerns 4

3.3 Component (UML) Diagram 4

3.4 Component Descriptions 5

3.5 Chosen Architecture Pattern and Rationale 5

Architecture Decision Record (ADR) Summary 5

3.6 Technology Stack & Data Stores 5

3.7 Risks & Mitigations 6

3.8 Traceability to Requirements 6

3.9 Security Architecture 6

4. Design 8

4.1 Design Overview 8

4.2 UML Sequence Diagrams 8

4.3 API Design 8

4.4 Error Handling, Logging & Monitoring 9

4.5 UX Design 9

4.6 Open Issues & Next Steps 9

5. Appendices 11

5.1 Glossary 11

5.2 References 11

5.3 Tools 11

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Voice Assistant for Multimedia Controls v1.0. It defines the system structure, component responsibilities, selected architecture pattern, technology choices, security architecture, design interfaces, error handling, logging, user interaction approach, and traceability to the approved Software Requirements Specification (SRS).

1.2 Scope

The system is a Raspberry Pi-based, voice-only multimedia control application. Spoken commands are captured through a USB microphone, converted to text using a speech-recognition engine, mapped to predefined multimedia intents, validated against an allow-list, and executed through a media player such as VLC. Supported functions are Play, Pause, Resume, Stop, Next Track, Previous Track, Volume Up, Volume Down, and Mute. Camera input, gesture recognition, and MediaPipe are outside the scope.

1.3 Audience

Developers, QA/test engineers, security reviewers, course instructors, project evaluators, and project team members.

1.4 Definitions

SRS – Software Requirements Specification; SAD – Software Architecture and Design Specification; STP – Software Test Plan; STT/ASR – Speech-to-Text / Automatic Speech Recognition; UI – User Interface; VLC – VideoLAN media player; Intent – normalized action derived from a spoken command; RTM – Requirements Traceability Matrix; ADR – Architecture Decision Record; Pi – Raspberry Pi.

2. Document Overview

2.1 How to use this document

This document is used as the architectural and design reference for implementation and testing. Section 3 describes the architecture, components, technology choices, risks, requirement traceability, and security architecture. Section 4 describes detailed design interfaces, sequences, error handling, logging, and user experience. Section 5 provides the glossary, references, and tools. The UML diagrams marked in Sections 3.3 and 4.2 shall be replaced by the final diagrams prepared by the team.

2.2 Related Documents

Document	Version / Reference	Purpose
Software Requirements Specification (SRS)	v1.0	Defines functional, non-functional, security, and acceptance requirements.
Software Test Plan (STP)	v1.0	Defines test approach, security validation, test cases, and SRS traceability.
Requirements Traceability Matrix (RTM)	Within SRS / STP	Maps requirements to design and test coverage.

3. Architecture

3.1 Goals & Constraints

Goals: Provide a modular, understandable, testable, reliable, and secure voice-control architecture; keep the multimedia control path local to the Raspberry Pi where practical; make each major processing stage independently testable; and support clear mapping from requirements to design components.

Constraints: Recognition quality depends on microphone quality and ambient noise; selected speech-recognition engine may introduce a network dependency; Raspberry Pi resources are limited compared with a desktop; only predefined multimedia intents may execute actions; raw microphone recordings shall not be stored during normal operation; and camera/gesture processing is out of scope.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
End User	Simple voice interaction, correct multimedia control, quick response, clear feedback.
Project Developers	Modularity, maintainability, integration clarity, hardware compatibility.
Tester / QA	Testability, measurable acceptance criteria, predictable error handling and traceability.
Security Reviewer	Command integrity, allow-list enforcement, privacy, log protection, prevention of arbitrary OS command execution.
Course Instructor / Evaluator	Alignment with SRS, clear architecture, demonstrable implementation, testing and Git/GitHub evidence.

3.3 Component (UML) Diagram

.

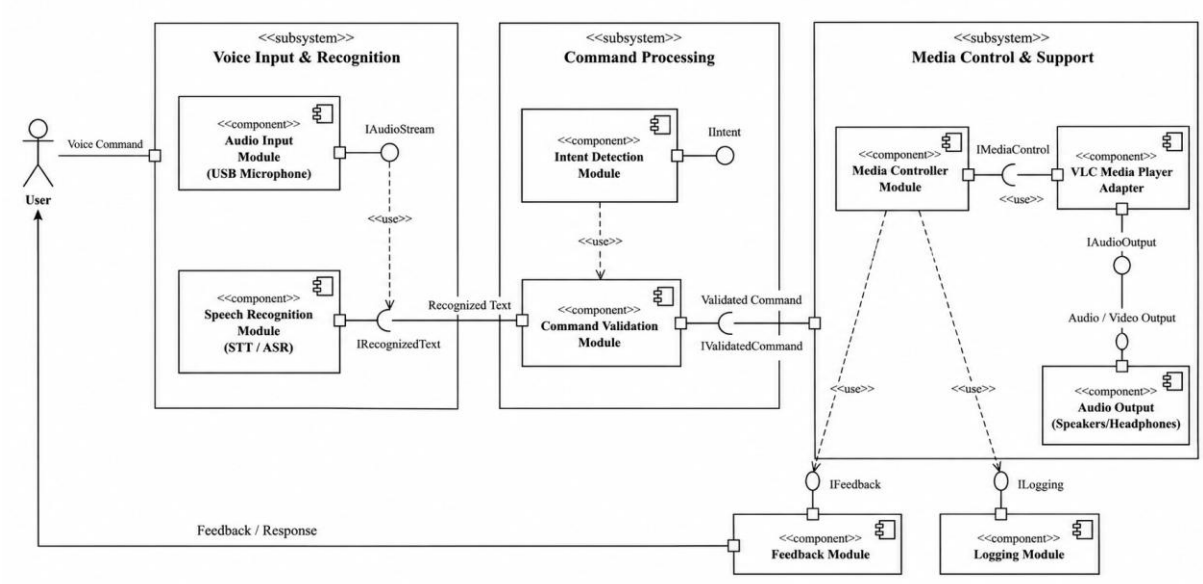


Figure 3.1: Component Architecture of the Voice Assistant

3.4 Component Descriptions

Component	Responsibility	Key Interfaces / Dependencies
USB Microphone / Audio Input	Captures the user's spoken command.	Audio input driver; Raspberry Pi USB/audio subsystem.
Speech Recognition	Converts captured speech into text.	Audio stream input; selected STT/ASR engine.
Intent Processor	Maps recognized text to a normalized supported intent.	Recognized text → IntentResult.
Command Validator	Checks intent validity, confidence/equivalent validation, and allow-list membership.	IntentResult → ValidationResult.
Media Controller	Translates validated intents into multimedia actions.	ValidationResult → MediaActionResult; VLC/player interface.
Feedback Module	Reports successful execution, errors, and status to the user.	UI/terminal output and/or voice/status feedback.
Logging Module	Records timestamped command, intent, and execution-result events without raw audio by default.	Local protected log storage.
Raspberry Pi Platform	Provides the runtime environment and coordinates local components.	Raspberry Pi OS/Linux, Python runtime, audio interfaces.

3.5 Chosen Architecture Pattern and Rationale

A layered modular architecture is selected. The system separates input, recognition, intent processing, validation, media control, and feedback/logging responsibilities. This separation keeps the command path understandable, makes unit and integration testing practical, and limits the effect of changes in one component on the rest of the system.

A microservices architecture is not selected for the current scope because the target system is a small, locally deployed Raspberry Pi application and the additional service orchestration would add complexity without a demonstrated requirement for distributed services.

The architecture is organized as: Input Layer → Speech Recognition Layer → Intent / Command Processing Layer → Validation Layer → Media Control Layer, with Feedback and Logging as supporting services.

Architecture Decision Record (ADR) Summary

ADR	Decision	Reason
ADR-01	Use Raspberry Pi as the central runtime platform.	Matches the hardware target and keeps the primary control path local.
ADR-02	Use voice-only interaction for this release.	Matches the approved SRS scope; camera, gesture recognition, and MediaPipe are excluded.
ADR-03	Use layered modular processing.	Separates responsibilities and supports unit/integration testing.
ADR-04	Restrict execution to predefined intents.	Protects command integrity and prevents arbitrary system-command execution.
ADR-05	Do not retain raw audio by default.	Aligns with privacy/security requirements in the SRS.

3.6 Technology Stack & Data Stores

Area	Technology / Choice	Purpose
Hardware	Raspberry Pi	Central processing and deployment platform.
OS	Raspberry Pi OS / Linux	Runtime operating environment.

Area	Technology / Choice	Purpose
Programming	Python 3.x	Application logic and component integration.
Speech Recognition	Selected STT/ASR engine/library	Speech-to-text conversion.
Media Control	VLC or compatible media player	Playback and volume operations.
Logging / Configuration	Local files with restricted access	Operational events and configuration.
Database	None in current scope	The SRS specifies local event logging rather than a database-backed design.

3.7 Risks & Mitigations

Risk	Mitigation
Background noise reduces recognition quality.	Use controlled demonstration conditions, microphone positioning, repeat trials, and accuracy testing.
Speech-recognition engine has network dependency.	Use stable connectivity or an offline engine if supported by the final implementation.
Raspberry Pi resource constraints.	Use lightweight modules, monitor CPU/memory during testing, and keep test media controlled.
VLC/player interface differences.	Record the tested media-player version/configuration and keep the integration layer isolated.
Microphone/driver failure.	Run a microphone smoke test before system testing and keep a known-working input path available where possible.

3.8 Traceability to Requirements

SRS Requirements	Architecture Element	Design Relationship
VA-F-001 to VA-F-002	USB Microphone + Speech Recognition	Capture and convert spoken input to text.
VA-F-003 to VA-F-004, VA-F-014	Intent Processor + Command Validator	Detect normalized intents, accept equivalent phrases, and validate supported commands.
VA-F-005 to VA-F-010	Media Controller	Execute play, pause, resume, stop, next, and previous operations.
VA-F-011 to VA-F-013	Media Controller / Volume Control	Execute volume up, volume down, and mute.
VA-F-015 to VA-F-016	Feedback + Error Handling	Provide success/error feedback and fail safely for invalid commands.
VA-F-017	Logging Module	Record timestamped command, intent, and execution-result events.
VA-F-018	Error Handler / Recovery Logic	Recover from recognition or media-player errors where possible.
VA-NF-001 to VA-NF-007	Cross-cutting architecture	Performance, accuracy, reliability, usability, privacy, and startup behavior are verified at system level.
VA-SR-001 to VA-SR-006	Command Validator + Privacy + Config Access + Secure Logging	Implements allow-list, confidence validation, privacy, OS-command isolation, configuration protection, and secure logging.

3.9 Security Architecture

The security architecture follows the security objectives and security requirements in SRS Section 5.1. The command path is treated as untrusted input until intent validation succeeds.

Threat Category	Relevant Threat	Architectural Control
Spoofing	Untrusted voice input or unauthorized configuration access.	Do not use recognized speech as an authentication mechanism; restrict configuration access through operating-system/user permissions.

Threat Category	Relevant Threat	Architectural Control
Tampering	Modification of command mappings or security configuration.	Protect configuration files with restricted permissions and validate configuration at startup.
Information Disclosure	Exposure of raw audio or sensitive information in logs.	Do not retain raw audio by default; exclude passwords, tokens, and raw audio from logs.
Denial of Service	Repeated invalid input, recognizer failure, or media-player failure.	Use bounded processing, error handling, timeouts where applicable, and recovery logic.
Elevation of Privilege	Recognized text attempts to invoke arbitrary operating-system commands.	Map text only to predefined intents; never pass recognized text directly to a shell/OS command interface.

Security design principle: voice input may propose an action, but only the validated intent layer is permitted to request a multimedia operation.

4. Design

4.1 Design Overview

The runtime flow is designed as a controlled pipeline: the user speaks a command; the USB microphone provides audio; the speech-recognition component converts audio to text; the intent processor maps text to a supported intent; the command validator confirms that the intent is allowed; the media controller executes the action; and the system provides feedback and records the result. Components communicate through explicit internal interfaces so that each stage can be unit tested independently and integrated end-to-end.

4.2 UML Sequence Diagrams

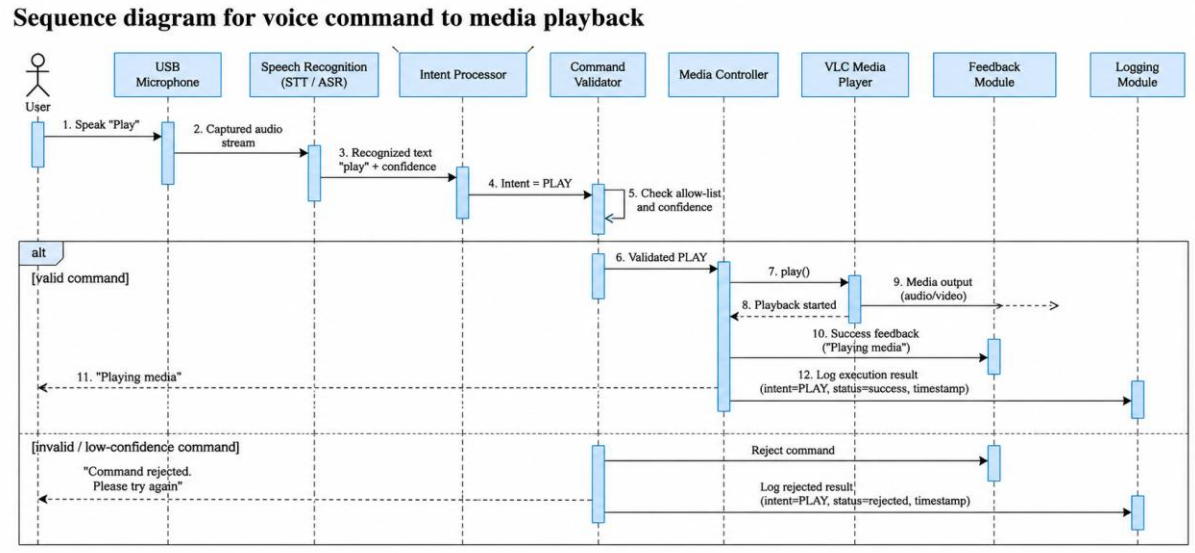


Figure 4.1: Voice Command to Media Playback Flow

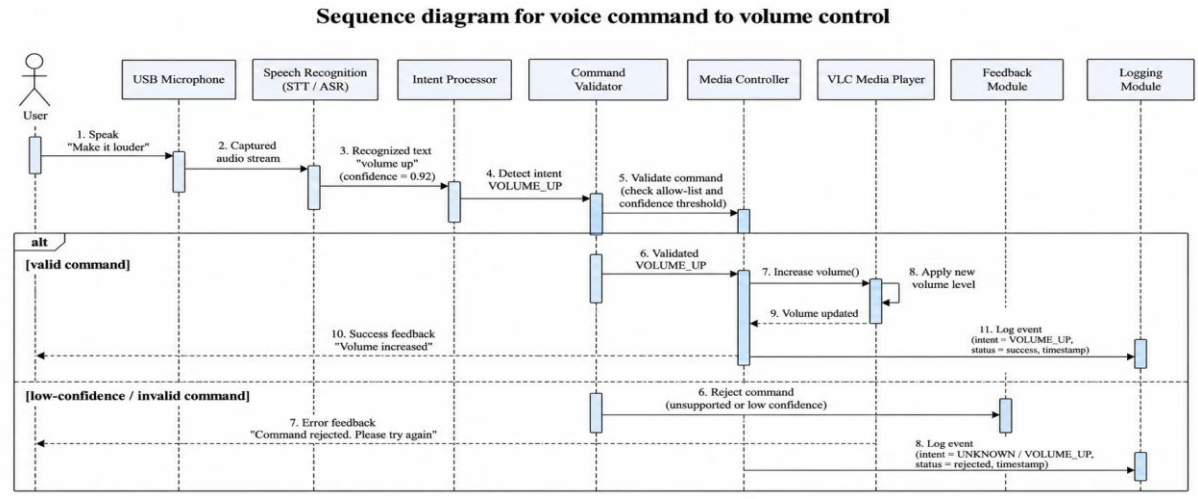


Figure 4.2: Voice Command to Volume Control / Validation Flow

4.3 API Design

The application uses internal module interfaces rather than a network REST API in the current scope. Each interface defines the expected input and output contract between components.

Interface	Operation	Input	Output	Error / Validation
Speech Recognition Interface	recognize(audio_input)	Captured audio stream / buffer	RecognitionResult {text, confidence}	Recognition failure, timeout, empty result.
Intent Processing Interface	detect_intent(text)	Recognized text	IntentResult {intent, parameters}	Unsupported/unknown phrase returns no executable intent.
Command Validation Interface	validate(intent_result)	IntentResult	ValidationResult {allowed, intent}	Rejects unsupported or low-confidence input.
Media Controller Interface	execute(validated_command)	Validated command	MediaActionResult {success, status}	Player unavailable, invalid state, unsupported action.

4.4 Error Handling, Logging & Monitoring

Error Handling: Errors shall be handled at the component boundary where they occur. Recognition failures shall return a controlled error/empty result; unsupported intents shall not trigger a media action; media-player failures shall produce an error status and invoke recovery where possible.

Logging: The logging module shall record timestamp, recognized intent, and execution result as defined by the SRS. Raw microphone audio shall not be stored during normal operation. Logs shall not contain passwords, tokens, or other sensitive credentials.

Monitoring: During development/testing, the team shall monitor command response time, intent accuracy, application stability, resource use, and failure counts. These measurements correspond to the SRS non-functional requirements and the STP metrics.

4.5 UX Design

The primary user interface is voice. The user speaks a supported command without navigating a complex visual interface. During development and evaluation, a terminal or lightweight interface may show the recognized text, detected intent, execution status, and errors. Feedback should be understandable and provided promptly. The system should not expose internal exceptions or sensitive details to the user.

UX Event	User-visible Behavior
Valid command	Execute the requested media action and provide success feedback.
Unrecognized / unsupported command	Provide a clear error/unsupported message and take no unintended action.
Low-confidence command	Reject or request another command according to the implementation's validation policy.
Media-player error	Show an understandable failure message and attempt recovery where possible.
Startup	Indicate readiness when the voice-control application is ready to accept commands.

4.6 Open Issues & Next Steps

- Finalize the speech-recognition engine/library used in the Raspberry Pi deployment and record whether the final mode is offline or network-dependent.
- Finalize the exact VLC/media-player control interface and version used for evaluation.
- Implement the first end-to-end command path and corresponding unit/integration tests.
- Configure the GitHub Actions workflow so every push runs automated software tests.
- Complete and validate the UML component and sequence diagrams against the implemented code.

- Run Sprint 1 performance, accuracy, error, security, and reliability tests and update the RTM with actual results.

5. Appendices

5.1 Glossary

Term	Definition
SAD	Software Architecture and Design Specification.
SRS	Software Requirements Specification.
STP	Software Test Plan.
STT / ASR	Speech-to-Text / Automatic Speech Recognition.
Intent	Normalized action derived from recognized voice text.
VLC	VideoLAN media player used for multimedia control.
RTM	Requirements Traceability Matrix.
ADR	Architecture Decision Record.
UAT	User Acceptance Testing.

5.2 References

- Team 10 Software Requirements Specification (SRS), v1.0, Voice Assistant for Multimedia Controls.
- Team 10 Software Test Plan (STP), v1.0, Voice Assistant for Multimedia Controls.
- IEEE 42010 – Architecture description guidance used as a general reference for architecture documentation.
- OWASP guidance for secure application design and input validation.
- NIST SP 800-160 – Systems Security Engineering principles, used as a general reference for security design considerations.

5.3 Tools

Tool	Purpose
PlantUML	Creation of UML component and sequence diagrams.
draw.io / diagrams.net	Alternative diagramming and editing tool.
Git / GitHub	Version control, collaboration, issues, pull requests, and CI/CD workflow hosting.
Jira Scrum	Sprint planning, task assignment, project tracking, and GitHub-linked development activity.
Python test framework (e.g., pytest)	Unit and integration test execution.
GitHub Actions	Automated build/test workflow on repository changes.
Raspberry Pi OS / Linux tools	Runtime setup, diagnostics, and hardware/software integration testing.